Lec 5

21 Jul, 2026

Recursion

What is Recursion?

Recursion is a technique where a **function calls itself** to solve a smaller instance of the same problem, until the problem becomes small enough to be solved directly.

Every recursive function must have two essential parts:

1. Recursive Case

The part of the function where it **calls itself**, but with a smaller or simpler input than before. This is what moves the problem closer to being solved.

2. Base Case

The **terminating condition** — the point at which the function stops calling itself and returns a direct answer. Without a base case, the function would call itself forever, causing a **stack overflow**.

General structure:

```
returnType recursiveFunction(parameters) {
    if (base_condition) {
        return base_result;      // base case
    }
    return recursiveFunction(smaller_input);  // recursive case
}
```

How Recursion Uses the Call Stack

Every time a function is called (including calling itself), the computer pushes a new **stack frame** onto the **call stack**. This frame stores:

- The function's local variables
- The parameters passed in
- The return address (where to resume execution after this call finishes)

As recursive calls keep happening, frames keep getting **pushed** on top of each other. Once the base case is hit, the function starts **returning** — and each frame is **popped off** the stack in reverse (Last In, First Out — LIFO) order, with each returning call using the result of the call above it to compute its own result.

This is why recursion works so naturally for problems that have a "wait for the smaller result, then combine" structure — the stack automatically remembers where each call needs to resume.

---

1. Factorial Using Recursion

Problem

Factorial of `n` (written `n!`) is defined as:

```
n! = n × (n-1) × (n-2) × ... × 1
0! = 1   (defined base case)
```

This has a natural recursive definition:

```
n! = n × (n-1)!
```

C++ Code

```cpp
#include <iostream>
using namespace std;

int factorial(int n) {
    // Base case
    if (n == 0 || n == 1) {
        return 1;
    }
    // Recursive case
    return n * factorial(n - 1);
}

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
```

```
    if (num < 0) {
        cout << "Factorial is not defined for negative numbers." << endl;
    } else {
        cout << num << "! = " << factorial(num) << endl;
    }
    return 0;
}
```

Detailed Explanation of Recursion Here

Let's trace `factorial(4)`:

1. `factorial(4)` → is `4 == 0 or 1`? No → returns `4 * factorial(3)`
2. `factorial(3)` → is `3 == 0 or 1`? No → returns `3 * factorial(2)`
3. `factorial(2)` → is `2 == 0 or 1`? No → returns `2 * factorial(1)`
4. `factorial(1)` → is `1 == 0 or 1`? **Yes** → returns `1` (base case reached, recursion stops going deeper)

Now the calls **unwind** (return back up):

5. `factorial(1)` returns `1`
6. `factorial(2)` = `2 * 1` = `2`
7. `factorial(3)` = `3 * 2` = `6`
8. `factorial(4)` = `4 * 6` = `24`

Final answer: `factorial(4) = 24`

How the Stack is Used Here

Each call to `factorial()` is pushed onto the stack **before** it can return, because each call depends on the result of the *next* call:

```
Stack grows downward (top = most recent call):

| factorial(1) → returns 1        |  <- base case, top of stack
| factorial(2) → waiting for f(1) |
| factorial(3) → waiting for f(2) |
| factorial(4) → waiting for f(3) |  <- bottom (original call)
```

- **Pushing phase (recursive case):** Each call `factorial(n)` cannot compute `n * factorial(n-1)` until `factorial(n-1)` returns. So the function "pauses" mid-execution, and a new frame for `factorial(n-1)` is pushed on top. This keeps happening until `n == 1`.

- **Popping phase (base case reached, unwinding begins):** `factorial(1)` returns `1` immediately — its frame is popped. This value is handed back to `factorial(2)`, which multiplies it by `2`, returns `2`, and its frame is popped. This continues until the original call `factorial(4)` finally computes and returns `24`.

Important: If there were no base case, `factorial(n-1)` would keep calling `factorial(n-2)`, `factorial(n-3)`, ... forever (or until negative infinity), pushing infinite frames onto the stack, eventually causing a **stack overflow** (program crash).

---

2. Tower of Hanoi

The Puzzle — Detailed Explanation

Tower of Hanoi is a classic puzzle involving:

- **3 rods**: Source (A), Auxiliary (B), Destination (C)
- **N disks** of different sizes, stacked on the source rod in decreasing size order (largest at the bottom, smallest at the top)

Goal: Move the entire stack from the **source rod** to the **destination rod**, using the auxiliary rod as helper.

Rules:

1. Only **one disk** can be moved at a time.
2. Only the **topmost disk** of a stack can be moved.
3. A **larger disk can never be placed on top of a smaller disk**.

Why Recursion Fits This Problem Naturally

To move `N` disks from Source → Destination, think of it in three big steps:

1. Move the **top (N-1) disks** from Source → Auxiliary (using Destination as helper)
2. Move the **largest (Nth) disk** directly from Source → Destination
3. Move the **(N-1) disks** from Auxiliary → Destination (using Source as helper)

Notice that steps 1 and 3 are **themselves Tower of Hanoi problems**, just with fewer disks (`N-1` instead of `N`) and different rod roles. This is the recursive case.

Base case: When there is only **1 disk** left to move — just move it directly from source to destination. No further breakdown needed.

Algorithm (Pseudocode)

```
Algorithm: TowerOfHanoi(n, source, auxiliary, destination)

1. If n == 1:
       Move disk 1 from source to destination
       Return   (base case)

2. TowerOfHanoi(n-1, source, destination, auxiliary)
       // move top n-1 disks out of the way, onto auxiliary

3. Move disk n from source to destination
       // move the largest remaining disk directly

4. TowerOfHanoi(n-1, auxiliary, source, destination)
       // move the n-1 disks from auxiliary onto destination
```

C++ Code

```cpp
#include <iostream>
using namespace std;

void towerOfHanoi(int n, char source, char auxiliary, char destination) {
    // Base case
    if (n == 1) {
        cout << "Move disk 1 from " << source << " to " << destination <<
endl;
        return;
    }

    // Recursive case: move top n-1 disks from source to auxiliary
    towerOfHanoi(n - 1, source, destination, auxiliary);

    // Move the nth (largest remaining) disk from source to destination
    cout << "Move disk " << n << " from " << source << " to " << destination
<< endl;

    // Recursive case: move the n-1 disks from auxiliary to destination
    towerOfHanoi(n - 1, auxiliary, source, destination);
}

int main() {
    int n;
    cout << "Enter number of disks: ";
    cin >> n;
```

```
    towerOfHanoi(n, 'A', 'B', 'C');  // A = source, B = auxiliary, C =
destination

    return 0;
}
```

Python Code

```
def tower_of_hanoi(n, source, auxiliary, destination):
    # Base case
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    # Recursive case: move top n-1 disks from source to auxiliary
    tower_of_hanoi(n - 1, source, destination, auxiliary)

    # Move the nth (largest remaining) disk from source to destination
    print(f"Move disk {n} from {source} to {destination}")

    # Recursive case: move the n-1 disks from auxiliary to destination
    tower_of_hanoi(n - 1, auxiliary, source, destination)


if __name__ == "__main__":
    n = int(input("Enter number of disks: "))
    tower_of_hanoi(n, 'A', 'B', 'C')  # A = source, B = auxiliary, C =
destination
```

Detailed Trace for n = 3 (Source=A, Auxiliary=B, Destination=C)

Calling `towerOfHanoi(3, A, B, C)`:

```
towerOfHanoi(3, A, B, C)
 ├── towerOfHanoi(2, A, C, B)          // move top 2 disks A→B, using C as
helper
 │    ├── towerOfHanoi(1, A, B, C)     // base case: move disk 1, A→C
 │    │      → "Move disk 1 from A to C"
 │    ├── "Move disk 2 from A to B"
 │    └── towerOfHanoi(1, C, A, B)     // base case: move disk 1, C→B
 │           → "Move disk 1 from C to B"
 ├── "Move disk 3 from A to C"
 └── towerOfHanoi(2, B, A, C)          // move top 2 disks B→C, using A as
helper
```

```
        ├── towerOfHanoi(1, B, C, A)     // base case: move disk 1, B→A
        │      → "Move disk 1 from B to A"
        ├── "Move disk 2 from B to C"
        └── towerOfHanoi(1, A, B, C)     // base case: move disk 1, A→C
               → "Move disk 1 from A to C"
```

Final output for n = 3:

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Total moves = **7** = `2^3 - 1` (for `n` disks, minimum moves required = `2^n - 1`).

How Recursion Is Applied — In Depth

- **Recursive case (two calls per level):** Unlike factorial (which makes one recursive call per level), Tower of Hanoi makes **two recursive calls** at each level — one before moving the big disk (`towerOfHanoi(n-1, source, destination, auxiliary)`), and one after (`towerOfHanoi(n-1, auxiliary, source, destination)`). Each call treats a *smaller sub-problem* (`n-1` disks) with the **roles of the rods swapped**, which is the clever part — the auxiliary rod becomes the new "destination" for the sub-problem, and vice versa.
- **Base case:** When `n == 1`, there's nothing left to break down — just move that single disk directly. This stops the recursive branching.
- **Stack behavior:** Because each call to `towerOfHanoi(n, ...)` makes **two further recursive calls**, the call tree is not a single chain (like factorial) but a **binary recursion tree**. The call stack still works in the same LIFO fashion — the leftmost/first recursive call must fully complete (all the way down to its base cases and back) before the "move disk n" print statement executes, and *that* must complete before the second recursive call begins. This is why the trace above prints moves in a specific left-to-right, depth-first order: the stack forces every branch to fully resolve before moving to the next.
- **Number of stack frames alive at once:** At the deepest point of recursion, there are `n` frames on the stack simultaneously (one for each disk level from `n` down to `1`), just like factorial — despite the recursion tree having `2^n - 1` total calls, the **maximum stack depth** at any instant is only `n`.

Summary Table

Concept	Factorial	Tower of Hanoi
Recursive calls per level	1	2
Base case	`n == 0` or `n == 1`	`n == 1`
Shape of recursion tree	Linear chain	Binary tree
Max stack depth	`n`	`n`
Total operations	`n` multiplications	`2^n - 1` moves